

Lecture 10: Reinforcement Learning – Q-Learning

Yaxin Bi
School of Computing
University of Ulster

1

Contents

- ▶ How does the environment transition the next state
- ▶ Deterministic and stochastic policies
- ▶ Relationship between reward, return and values
- ▶ Construct Q-values
- ▶ Grid-world example

2

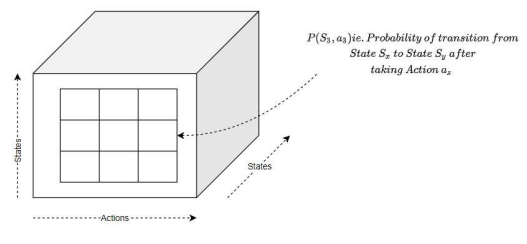
How does the Environment Transition the next State

- ▶ Given a current state, and the action picked by the agent, how does the environment figure out the next state and reward?
- ▶ In fact most environments have complex internal dynamics that control how they behave when an action is taken from a particular state.
 - ▶ Control drone navigation
 - ▶ CartPole problem
 - ▶ Mountain car problem
 - ▶ From which it depends on the laws of physics that control air flows, motion, thermodynamics
- ▶ In RL, the environment is usually treated a black box, which could be formulated as an MDP in form of a large transition probability matrix or function.

3

3

Model Transition Metrix



- ▶ Each of states with some probability, determining the next state, which is known as the Transition Probability.

4

4

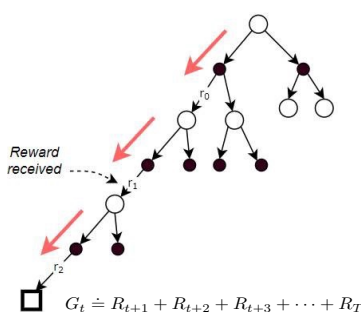
How Does the Agent Pick the Action

- ▶ On the other hand, how the agent decides what action to take in a given state, which is precisely the RL problem to be solved.
- ▶ Three concepts are used for determining an action below:
 - ▶ Return – total rewards over the time-steps
 - ▶ Policy – the strategy followed to pick the action
 - ▶ Value – informing expected return when following some policy

5

5

Return



- ▶ As the agent executes time-steps, it accumulates reward at each time-step.
- ▶ However, rather than any individual reward, RL really cares about is the cumulative rewards.

6

6

Return (cont'd)

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- Discount factor gamma (γ) – encouraging the agent to care more about the immediate reward compared to later rewards, as later rewards will be more heavily discounted.
- The agent has to give a weight to the reward now versus later; the immediate reward is more valuable.
- The agent has to choose between getting some reward now versus getting a much bigger reward later; the bigger reward is most likely preferable.
- The ultimate goal of the agent is to get the maximum Return, not just over one episode, but over many, many episodes.

7

7

Policy

- How the agent decides which action to pick in a given state. There can be many different strategies that an agent might use:
 - Pick the next action at random
 - Pick the next state that gives the highest known reward
 - Take chances and explore new states in the hope of finding a better path.
 - Play it safe and avoid the chance of a negative reward.
- Any strategy that the agent follows to decide which action to pick in a given state, is called a Policy.

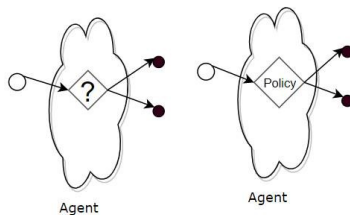
8

8

Policy (cont'd)

- a Policy is something that maps a given state to an action to be taken, or informs the agent which action to pick from a given state

$$\pi(a|s) = \mathbb{P}[A_t = a | S_t = s]$$



9

9

Lookup Table

- So given the current state, the agent looks up that state in the table to find the action that it should pick.
- For real-world problems, there are so many states and so many actions, that a function is used, not a Lookup Table, that maps a state to an action.

		Actions →		
		a_1	a_2	a_3
States ↑	s_1	$\pi(a_1 s_1)$	...	...
	s_2	...	...	...
	s_3	...	...	$\pi(a_3 s_3)$
	s_4	...	...	...

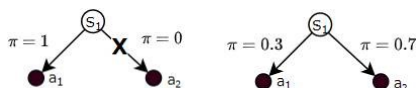
Probability of taking Action a_3 when in State s_3

10

10

Deterministic and Stochastic Policies

- A Deterministic Policy is a Policy that the agent always chooses the same fixed action when it reaches a particular state.
- A Stochastic Policy is one where the agent varies the actions it chooses for a state, based on some probability for each action.
- It might use this while playing a game, for instance, if it always played the same move, opponents can figure this out and easily defeat it.



11

11

Options of Policies

- The action that the agent takes from a given state determines the reward it obtains.
- Hence the goal of the agent is to pick the action that maximizes its Return.
 - i.e. the agent's goal is to follow a Policy (which is how it picks its actions) that maximizes its Return.
- There are more than one policy, which the agent could follow, which one could give it the highest Return.
- The agent needs to compare two Policies to decide which is better.
- Choice of policies is based on another notion of Value.

12

12

Value

- Scenario one: The agent starts from a state, and always picks actions based on a policy π , what would its average Return be over many episodes?
- This average long-term Return, or expected Return, is known as the Value of **that particular state**, under policy π .
- Scenario two: the agent could start from a **state-action pair** i.e. it has already taken a **particular action** from a **particular state**.
- If going forward from that state-action, it always picks actions based on the given policy π , what is the Return it could expect to get?

13

13

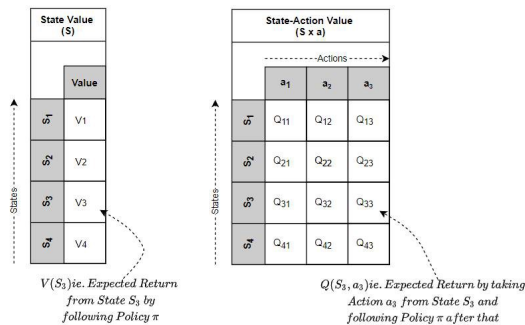
Value (cont'd)

- Value can be defined by a Lookup Table which maps a State or a State-Action pair to a Value. Thus we have two types of Value:
- State Value – the expected Return from a given state, by executing actions based on a given policy π from that state onward.
 - The State Value function maps a State to its Value.
- State-Action Value (Q-Value) – the expected Return by taking a given action from a given state, and then, by executing actions based on a given policy π after that.
 - The State-Action Value function maps a State-Action pair to its Value.

14

14

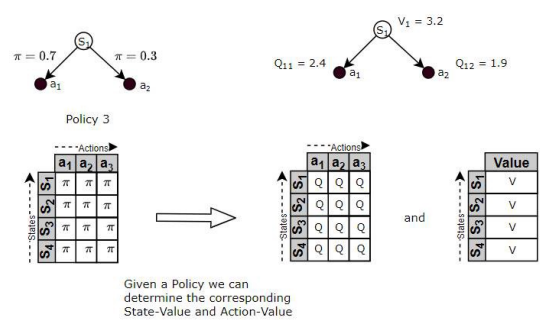
Comparison between Two Types of Values



15

15

Use the Value Function to Compare Policies



16

16

Relationship Between Reward, Return and Values

- Reward is the immediate reward obtained for a single action (one time-step).
- Return is the total discounted rewards obtained until the end of that episode (many time-steps).
- Value is the mean Return (aka expected Return) over many episodes.
- Intuitively, like a human, the agent learns from experience via interacting with the environment over episodes, it obtains the Returns for each episode.
- As it accumulates more experience (i.e. obtaining Returns over episodes), it gets a sense of which states, and which actions in those states yield the most Return.
- This 'experience' is regarded as 'Value'.

17

17

Taxonomy of Well-known RL Solutions

	Prediction	Control
Model-based Planning	Dynamic Programming Policy Evaluation	- Dynamic Programming Value Iteration - Dynamic Programming Policy Iteration
Model-free Reinforcement Learning	- Monte Carlo Prediction - Temporal Difference TD(0) - TD(λ) Backward	- Monte Carlo Control - Sarsa - Sarsa Backward - Q Learning - Deep Q Networks - Policy Gradient - Actor Critic

Most interesting real-world problems are Model-Free Control problems. This is what most people mean when they use the term 'Reinforcement Learning'

18

18

Q-Learning

- Q-Learning is the most interesting of the Lookup-Table-based approaches which is what Deep Q-Learning is based on.
- The Q-learning algorithm uses a Q-table of State-Action Values (also called Q-values).
 - This Q-table has a row for each state and a column for each action.
 - Each cell contains the estimated Q-value for the corresponding state-action pair.
- The algorithm starts by initializing all the Q-values to zero, and then the agent interacts with the environment and gets feedback,
 - iteratively improving these Q-values until they converge to the Optimal Q-values.
 - updating them by the Bellman equation.

19

19

How Construct a Q-table

- Let's take a simple grid game. Consider a 3x3 grid, where players start in the Start square and want to reach the Goal square as their final destination, where they get a reward of 5 points.



- Some squares are Clear while some contain Danger, with rewards of 0 points and -10 points respectively.
- In any square, the player can take four possible actions to move Left, Right, Up, or Down.

20

20

How Construct a Q-table (cont'd)

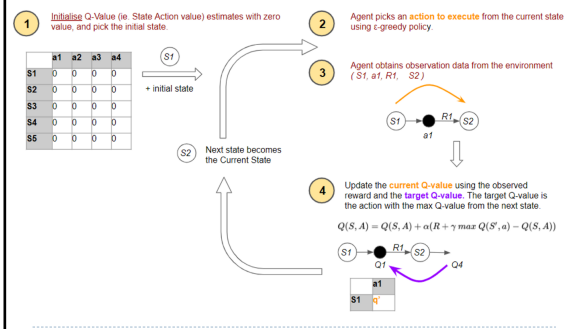
	Left	Right	Up	Down
(1,1)	0	0	0	0
(1,2)	0	0	0	0
(1,3)	0	0	0	0
(2,1)	0	0	0	0
(2,2)	0	0	0	0
(2,3)	0	0	0	0
(3,1)	0	0	0	0
(3,2)	0	0	0	0
(3,3)	0	0	0	0

- We use the Q-table to lookup the Q-value for any state-action pair.
- The value in a particular cell, for instance, say ((2, 2), Up) is the Q-value (or State-Action value) for the state (2, 2) and action 'Up'.

21

21

Q-Learning Algorithm

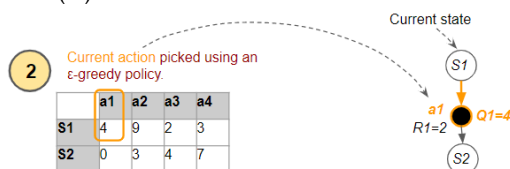


22

22

Details in Step #2

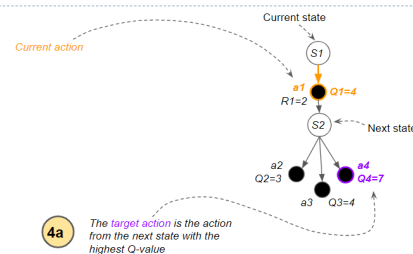
- In this step, the agent uses the ϵ -greedy policy to pick the current action (a1) from the current state (S1).
- Passing the action to the environment to execute and gets feedback in the form of a reward (R1) and the next state (S2).



23

23

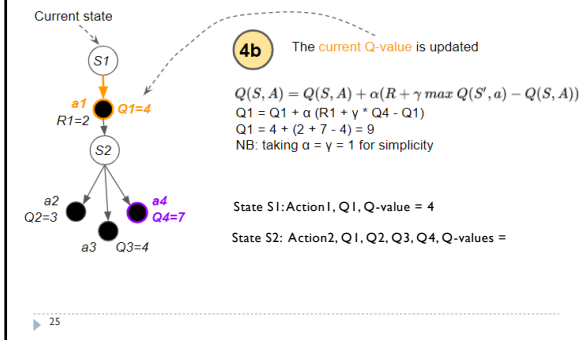
Details in Step #4a



24

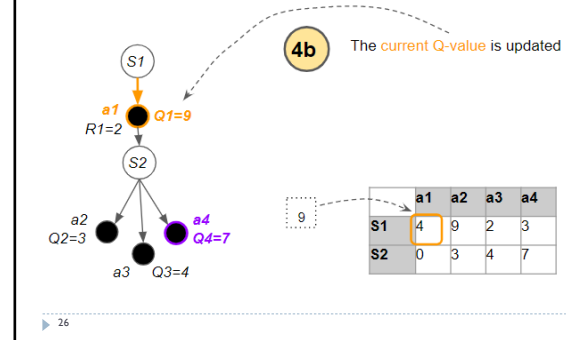
24

Details in Step #4b



25

Details in Step #4b (cont'd)



26

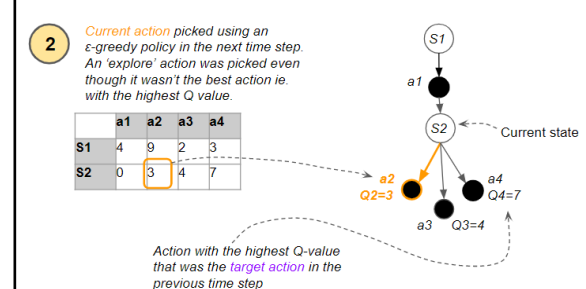
Explore and Exploitation

- There are two actions involved:
 - Current action – the action from the current state that is actually executed in the environment, and whose Q-value is updated.
 - Target action – has the highest Q-value from the next state, and used to update the current action's Q value.
- Q-learning explores and discovers new paths for actions to execute
- However, it updates Q-value estimates to improve learning experience, the algorithm always uses the best Q-value, even though that action may not get executed.

27

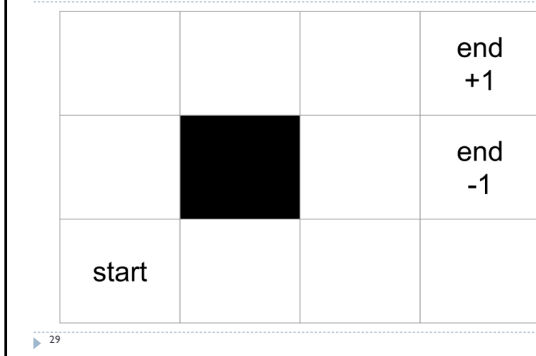
27

“Off-Policy” in Next Time-step



28

Example: Grid-world Game



29

Playing Rule

- The agent starts at the left bottom corner (the 'start' sign) and ends at either +1 or -1 reward.
- At each step, the agent has 4 possible actions, **up**, **down**, **left** and **right**, whereas the **Black Block** is a wall
- This game assumes that each action is deterministic, i.e. the agent will go where it intends to go. For instance, when the agent decides to take action up at (2, 0), it will land in (1, 0) rather than (2, 1) or elsewhere.
- The agent hit the wall, it will remain at the same position.

30

30

Implement Board and Reward

```
1 # global variables
2 BOARD_ROWS = 3
3 BOARD_COLS = 4
4 WIN_STATE = (0, 3)
5 LOSE_STATE = (1, 3)
6 START = (2, 0)
7 DETERMINISTIC = True
```

```
1 def giveReward(self):
2     if self.state == WIN_STATE:
3         return 1
4     elif self.state == LOSE_STATE:
5         return -1
6     else:
7         return 0
```

31

```
1 def nxtPosition(self, action):
2     """
3     action: up, down, left, right
4     """
5     0 | 1 | 2 | 3|
6     1 |
7     2 |
8     return next position
9     """
10    if self.determine:
11        if action == "up":
12            nextState = (self.state[0] - 1, self.state[1])
13        elif action == "down":
14            nextState = (self.state[0] + 1, self.state[1])
15        elif action == "left":
16            nextState = (self.state[0], self.state[1] - 1)
17        else:
18            nextState = (self.state[0], self.state[1] + 1)
19        # if next state legal
20        if (nextState[0] >= 0) and (nextState[0] <= 2):
21            if (nextState[1] >= 0) and (nextState[1] <= 3):
22                if nextState != (1, 1):
23                    return nextState
24    return self.state
```

32

Value Iteration

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)]$$

- Once the agent reaches end of the game, either reward +1 or reward -1
- The reward propagates in a backward fashion and eventually the estimated value of all states along the way will be updated based on the formula above.
- Assume there are 2 states, S1 and S2 with initial estimated value 0,
- The agent moves from S1 to S2 and gets reward 1, then the new estimate of S1 = S1 + $\alpha(S2 - S1)$, which is 0 + 0.1(1-0) = 0 (assume α is 0.1 and reward at S1 is 0)

33

```
1 def play(self, rounds=10):
2     i = 0
3     while i < rounds:
4         # to the end of game back propagate reward
5         if self.State.isEnd:
6             # back propagate
7             reward = self.State.giveReward()
8             # explicitly assign end state to reward values
9             self.state_values[self.State.state] = reward # this is optional
10            print("Game End Reward", reward)
11            for s in reversed(self.states):
12                reward = self.state_values[s] + self.lr * (reward - self.state_v
13                self.state_values[s] = round(reward, 3)
14            self.reset()
15            i += 1
16        else:
17            action = self.chooseAction()
18            # append trace
19            self.states.append(self.State.nxtPosition(action))
20            print("current position {} action {}".format(self.State.state, action))
21            # by taking the action, it reaches the next state
22            self.State = self.takeAction(action)
```

34

Determine an Action

```
1 def chooseAction(self):
2     # choose action with most expected value
3     mx_nxt_reward = 0
4     action = ""
5
6     if np.random.uniform(0, 1) <= self.exp_rate:
7         action = np.random.choice(self.actions)
8     else:
9         # greedy action
10        for a in self.actions:
11            # if the action is deterministic
12            nxt_reward = self.state_values[self.State.nxtPosition(a)]
13            if nxt_reward >= mx_nxt_reward:
14                action = a
15            mx_nxt_reward = nxt_reward
16    return action
```

35